# Ders 13 Genişletilmiş Notu: Görsel Anlambilim ve Kelimelerden Cümle Üretimi Sunumları

## Ders 13 Genişletilmiş Notu: Görsel Anlambilim ve Kelimelerden Cümle Üretimi Sunumları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Görsel Anlambilime Giriş
- 2. Resim Tanımı Üretme: Bir Moddan Başka Bir Moda Çeviri
- 3. Show and Tell
- 4. Önceden Eğitilmiş Modeller, Örüntüler ve Varyans
- 5. Tanım Üretimi: Örnekleme ve Işın Araması
- 6. Video Tanımlama
- 7. Görsel Soru Cevaplama ve Yanlılık
- 8. Veri Kümesi Oluşturmanın Zorluğu ve Kısa Yol Öğrenme
- 9. Karşıtsal Öğrenme ve SimCLR
- 10. Resimleri Anlamanın Sınırları
- 11. Ödev 3 Sunumları: Üç Anahtar Kelimeden Cümle Üretimi
- Örnek Sorular
- Dersin Toparlaması


### 1. Görsel Anlambilime Giriş

Bu derste modellerin işlediği içeriğe metinlerin yanında resimler de girdi. Hoca bu yüzden slaytlarına “görsel anlambilim” adını verdi (slayt dosyası “bolum7”, 21 slayt). Plan şuydu: resim tanımı üretme (image captioning), onun küçük bir parçası olarak video tanımlama (video captioning), görsel soru cevaplama ve üretici modellerin ön eğitiminde yaygın kullanılan bir teknik olan karşıtsal öğrenme (00:11:07). Dersin ikinci yarısı üçüncü ödevin sunumlarına ayrıldı.

Resim tanımı üretmeyle ilgili eski çalışmalar cümle üretmekten çok resimde neler olduğunu yakalamaya çalışır. Bunlarda CNN tarzı yaklaşımlar kullanılır ve nesnelerin birlikte bulunma bilgisi öğrenilmeye çalışılır. Hocanın örneği: resimde bir kaşık varsa, yanındaki çatala benzeyen şeklin çatal olma olasılığı daha yüksektir (00:12:43).

### 2. Resim Tanımı Üretme: Bir Moddan Başka Bir Moda Çeviri

Image captioning’de modele bir resim verilir, model o resmin içinde ne olduğunu metin olarak oluşturur (00:13:24). Hoca bunu uçtan uca (end-to-end) bir yapı olarak düşününce bir çeviri sistemi olarak gördü: bir dilden başka bir dile değil, bir moddan başka bir moda çeviri. Resim olan bir şey metne çevrilir. Tersi de mümkündür; bugün bir metin verip onu anlatan bir resim üreten birçok sistem vardır.

Bu yüzden yapı bir kodlayıcı (encoder) ve bir çözücüden (decoder) oluşur:

- **Kodlayıcı:** girdi modundaki veriyi alır ve gizli bir uzaya (latent space) taşır. Bu uzay insanlar için çok anlamlı değildir.

- **Çözücü:** gizli uzaydaki vektörü istenen biçime, istenen moda çevirir.

İngilizceden Türkçeye çeviri, text-to-text, image-to-text ve image-to-image yapılarının hepsinde mantık budur. Hocaya göre kodlayıcı-çözücü mimarisi bugün en yaygın yaklaşımlardan biridir ve kodlayıcıyla çözücü için en sık kullanılan yapılar derin öğrenme mimarileridir (00:15:13).

## 3. Show and Tell

### 3.1 Mimari

Hoca 2014 tarihli Show and Tell çalışmasıyla başladı (ekrandan, 00:13:20; slaytta kaynak arXiv 1411.4555). Slayttaki şekilde bir pazar resmi "Vision Deep CNN"e verilir, CNN'in çıkışında bir vektör oluşur, bu vektör "Language Generating RNN"e verilir ve RNN şu metni üretir: "A group of people shopping at an outdoor market. There are many vegetables at the fruit stand." Hoca iki blok arasındaki vektörü kırmızıyla işaretleyip bunun az önce bahsettiği gizli uzay olduğunu söyledi. CNN kodlayıcı, RNN çözücüdür (00:16:00).

> **Ek bilgi (derste anlatılmadı):** Show and Tell, büyük olasılıkla Google'dan Vinyals ve arkadaşlarının "Show and Tell: A Neural Image Caption Generator" adlı çalışmasıdır. Resim tanımı üretmede CNN kodlayıcı ve LSTM çözücü birleşimini yaygınlaştıran ilk çalışmalardandır.

### 3.2 Optimizasyon ve otoregresif üretim

Slayttaki maddeler şunlardır (ekrandan, 00:20:00):

- 2. yaklaşım: uçtan uca bir sistem
- Optimizasyon: $\theta^\star = \arg\max_\theta \sum_{(I,S)} \log p(S \mid I; \theta)$
- $I$: resim, $S$: resmin doğru tanımı, $\Theta$: model parametreleri
- $N$ uzunluğundaki bir tanımın olasılığı: $\log p(S \mid I) = \sum_{t=0}^{N} \log p(S_t \mid I, S_0, \ldots, S_{t-1})$

Hoca formülü şöyle açıkladı: elinizde (resim, tanım) ikilileri vardır, 50 ya da 100 tane, ne kadarsa. Bütün resimler için doğru tanımları üretmeye çalışırsınız ve bunu en başarılı yapabilen $\theta$'yı ararsınız. Buradaki $\theta$, CNN ile RNN'in hepsinin birden parametreleridir (00:17:01).

Tanımın olasılığı ise metnin tek seferde üretilmediğini gösterir. Hocaya göre bugünkü mimarilerin çok büyük kısmı metni tek seferde üretmez, adım adım üretir. Bu tam bir otoregresif modeldir: o ana kadarki kısım ($S_0$'dan $S_{t-1}$'e) ve resim verildiğinde bir sonraki kelime $S_t$ tahmin edilir. Örnekte "outdoor"a kadar gelindiyse sonraki adımda "market" üretilmesi istenir. Bu, bir sonraki şeyin tahmini (next token prediction) problemidir; o şey bir token ya da başka bir birim olabilir (00:18:30).

### 3.3 Ayrıntılar: LSTM ve kelime gömmeleri

Sonraki slayt mimariyi açar (ekrandan, 00:21:00):

- CNN çıkışı LSTM'e giriştir.
- CNN parametreleri önceden eğitilmiş (pre-trained) değerlerle başlar.
- $S_x$: one-hot vektör (slaytta "one hat vektör" yazıyor).
- $W_e$: kelime gömme katmanı (word embedding layer).
- 512 LSTM hücresi vardır.

Şekilde resim CNN'den geçip ilk LSTM'e verilir; ardından her adımda $W_e S_0, W_e S_1, \ldots, W_e S_{N-1}$ LSTM'e girer ve her adımın çıkışında $p_1, p_2, \ldots, p_N$ olasılıkları ile $\log p_1(S_1), \log p_2(S_2), \ldots, \log p_N(S_N)$ hesaplanır.

Hoca slaytta "RNN" yazılıp şekilde "LSTM" çizilmesini şöyle açıkladı: RNN'in standart, klasik bir hali vardır, bir de LSTM ve GRU gibi versiyonları vardır. Her LSTM ya da GRU aslında bir RNN'dir (00:20:13). Şekildeki ilk sütun kodlayıcı bloktur; içindeki şekil bir CNN mimarisidir (ResNet ya da başka bir şey), yani önceden eğitilmiş bir modeldir (00:20:50).

Hoca şeklin bazı eksikleri olduğunu da belirtti: $S_t$'yi üretmek için $S_0$'dan $S_{t-1}$'e kadarki bilginin hepsi kullanılır, ama şekil görüntüyü dağıtmamak için bunu göstermemiştir (00:26:11). Bugün soldaki kodlayıcı ve sağdaki çözücü her iki tarafta da Transformer bloklarına dönüşmüştür. Zamanla RNN tabanlı yapılar Transformer'larla birleşerek tekrar ortaya çıkabilir; o tarz çalışmalar da vardır (00:26:34). Hocaya göre değişmeyen şey kodlayıcı-çözücü yapısıdır (00:19:49).

## 4. Önceden Eğitilmiş Modeller, Örüntüler ve Varyans

### 4.1 Önceden eğitilmiş model nedir

Önceden eğitilmiş model (pre-trained model), sizin göreviniz için değil, başka şeyler için, çok büyük miktarda veriyle, jenerik olarak eğitilmiş bir modeldir. Siz bunu kendi probleminize ince ayarla (fine-tune) uyarlarsınız (00:21:03).

### 4.2 Örüntülerin örüntüsü

Hoca derin öğrenme mimarilerinin ne yaptığını şöyle hayal etmeyi önerdi: ilk katman örüntü (pattern) öğrenir, bir sonraki katman örüntülerin örüntüsünü, bir sonraki örüntülerin örüntülerinin örüntüsünü öğrenir. Katmanlar yükseldikçe daha üst seviye, daha soyut birleşimler öğrenilir. Bu, resimler ve belli ölçüde metinler için geçerlidir (00:21:54).

Alt seviyedeki örüntüler eldeki problemden oldukça bağımsızdır: bir resimdeki köşeleri, kırılma noktalarını, parlak ya da koyu bölgeleri bulmak problem alanından bağımsızdır. Bu yüzden alt katmanlar alan, veri kümesi ve görevden bağımsızdır (domain agnostic, dataset agnostic, task agnostic). Yukarı çıktıkça probleme özel işlemler gerçekleşir. Önceden eğitilmiş modeller alt katmanları çok büyük veriden öğrendiği için oldukça gürbüzdür (robust) (00:22:42).

### 4.3 Büyük model, az veri ve varyans

Çok büyük bir modeli çok az veriyle eğitirseniz eğitebilirsiniz, ama girdideki küçük bir değişiklik çıktıda büyük değişikliğe yol açabilir. Hoca bunun, modelin varyansının yüksek olduğunu söylemenin başka bir yolu olduğunu söyledi. Veri miktarı arttıkça varyans azalır. Büyük bir modelin varyansını en aza indirmek için büyük miktarda veri gerekir. Elinizde çok veri yoksa önceden eğitilmiş model kullanmak mantıklıdır (00:24:06).

Önceden eğitilmiş model yerine daha küçük, çok derin olmayan bir model kullanmak ise daha az sofistike, somuta daha yakın örüntülerle çalışmak demektir. Mimari derinleştikçe daha soyut örüntülerle iş görülür. Problem yüksek seviyeli örüntüler gerektiriyorsa büyük model kullanılmalıdır; bunun için de ya büyük veri ya da en azından varyansı düşürmek için önceden eğitilmiş bir model gerekir (00:24:42).

> **Hocanın vurgusu:** Elinizde büyük miktarda veri yoksa, en azından varyansı düşürmek için önceden eğitilmiş bir model kullanmak çok makuldür.

## 5. Tanım Üretimi: Örnekleme ve Işın Araması

Resim CNN'den geçince gizli uzayda bir nokta elde edilir. Bu nokta LSTM'e verilir ve LSTM adım adım her kelimeyi üretir (00:25:43). Slayttaki tanım üretimi seçenekleri şunlardır (ekrandan, 00:27:10):

- **Sampling (örnekleme):** $p_1$'e göre kelime seç, sonra $p_2$, $p_3$, ...
- **Beam search (ışın araması):** o ana kadarki en olası 20 cümleyi tut. Olasılık hesabı: $S = \arg\max_{S'} p(S' \mid I)$.
- Üretilen 20 cümleden en olası olanı seçildiğinde %80'i eğitim kümesindeki cümlelerdendir.

Hoca son maddeyi, sistemin çok çeşitli (diverse) şeyler üretemediğinin bir ifadesi olarak yorumladı: modelin ürettiği cümleler büyük ölçüde eğitim kümesinden çıkıyor. Bu da yeni resimlere uygun cevap verebilme yeteneğini sorgulatır. Hepsi değil, %80; ve bu 2014'teki bir çalışmadır. Bugün yeni durumlara uyum yeteneği çok yukarılara çekilmiştir; sistemin başarılı olması için girdi resimlerinin eğitim kümesine yakın olması gerekmez (00:27:14).

Sonuç slaytında o dönem için oldukça başarılı tanımlar vardır. Hatalardan biri kabul edilebilir bir hatadır: üzerine bir sürü şey yapıştırılmış bir yer için sistem “buzdolabı” demiştir; ilk bakışta alakasız görünse de o bölge buzdolabı kapaklarını andırır. Başka bir örnekte “Yellow School Bus” denmiştir: bir araç olduğu doğrudur, süslenmiş olması “school” kısmını açıklayabilir. Hocaya göre bu hatalar çok alakasız şeyler değildir (00:28:42).

## 6. Video Tanımlama

Videolar için de çalışmalar vardır ve bunlar da yine çeviri sistemi, yani kodlayıcı-çözücüdür (00:29:26). Slayttaki açıklama: “Yine çeviri sistemi tabanlı. 2D filtrelerle resimlerden, 3D filtrelerle video küplerinden (resimlerin arkaya doğru dizili hali) özellik çıkarıyor.” (ekrandan, 00:30:00; slaytta kaynak CVPR 2017, Pan ve arkadaşları). Şekildeki akış şöyledir:

```
input video -> frames  -> image MIL model -> (young, child, floor, little) -> A_i --\
            -> video   -> video MIL model -> (cleaning, doing, machine, boy) -> A_v --> Transf
input sentence "a boy is cleaning the floor" -> w_t ----------------------------------------
```

Video bir kareler dizisidir. Karelerin tamamından bir vektör üretilir, sonra model videonun içeriğini (“şu oluyor, bu oluyor”) cümle olarak bulmaya çalışır (00:29:52).

Hocaya göre bu alanda başarılı çalışmalar olsa da resimler kadar ileri gidilmemiştir, çünkü çıkış uzayı çok daha büyüktür. Bir resmi anlamakla bir videoyu anlamak aynı zorlukta değildir: işin içine zaman bilgisi girer, niyetleri okumaya çalışmak gerekir. Bir sürü durumu tek bir resimle anlatmak zordur, videolar ise çok daha fazla kavram içerebilir. Resim tanımak için 20 katman yetiyorsa, videoda gereken soyutluk miktarı çok daha fazladır (00:30:26). Hoca, mavi-beyaz, yalnız çizgilerden oluşan eski bir çizgi filmi örnek verdi: tek bir resimle anlatılması çok güç olan şeyleri zaman bilgisiyle anlatabiliyordu (00:31:30).

Hocaya göre bu yapılarda bizim kavram dediğimiz şeyler arka tarafta yalnızca örüntülerden oluşur; “belki bizde de öyledir, bilmiyorum” diye ekledi. Video daha soyut bilgi gerektirdiği için videolarla iş yapmak daha zordur (00:32:20).

## 7. Görsel Soru Cevaplama ve Yanlılık

### 7.1 Görev

Soru cevaplama sistemlerinde bir soru sorulur ve cevap metnin içinden, başka bir yerden ya da seçenekler arasından bulunur. Görsel soru cevaplamada (Visual Question Answering, VQA) ise sorular bir resim hakkındadır (00:32:52). Bunun için büyük derlemler oluşturulmuştur (slaytta kaynak visualqa.org).

### 7.2 Resmi görerek ve görmeden verilen cevaplar

Slayttaki örneklerde her resim için sorular ve iki grup cevap vardır; altta “Yeşiller resmi görerek, maviler görmeden verilen cevaplar” yazar (ekrandan, 00:33:30). Örneğin “What color is the hydrant?” sorusuna resmi görerek “white and orange”, görmeden “red” ve “yellow” gibi cevaplar; “What does the sign say?” sorusuna her iki durumda da çoğunlukla “stop” verilmiştir.

Hoca deneyi şöyle açıkladı: yeşil kısımda sistemin girdisine resim de verilir ve sistem ona göre cevap verir. Aynı eğitilmiş modele test örneği olarak resim yerine boş bir resim verilir ve bu durumda verdiği cevaplara bakılır. Modelin mantıklı bir şey yaptığını söyleyebilmek için, resmi gördüğü ve görmediği durumdaki cevapların farklı olması beklenir (00:33:58). Bazı durumlarda mantıklı sonuçlar vardır, bazılarında hiç yoktur. Hoca bunu bAbI veri kümelerinde soruyu görmeden cevap üretebilme durumuna benzetti (00:34:36).

### 7.3 Eleştiriler

Slayttaki eleştiriler şunlardır (ekrandan, 00:40:00):

- Ancak VQA veri kümesinde:
- “What sport is” ile başlayan soruların %41’inin cevabı “tenis”.
- “How many” ile başlayan soruların %39’unun cevabı “2”.

- “Bu resimde balık var mı?” sorusu ancak resimde balık varsa soruluyor. “Bu resimde X var mı?” sorularının %81’inin cevabı “evet”.

Hoca bunların yanlılık (bias) olduğunu söyledi: böyle bir soru geliyorsa resme bakmana gerek yok, “evet” de, tutturma ihtimalin %80’dir. Makineler bu tür düzenlilikleri çok kolay yakalar (00:38:29).

### 7.4 Veri kümesinde güncelleme

Düzeltme için veri kümesi güncellenmiştir. Slayttaki maddeler (ekrandan, 00:38:55; kaynak arXiv 1612.00837):

- Sorularla cevaplar arasındaki düzenlilikleri ortadan kaldırmak için
- Resme bakmadan doğru cevap verebilmeyi önlemek için
- Bu tarz sistemlerin başarılarının abartılmaması için
- Aynı sorunun farklı resimler için farklı cevaplarının olduğu bir benchmark
- Yeterli mi? Başka ne yapılabilir?

Slayttaki örnek çiftler: “Who is wearing glasses?” (man / woman), “Where is the child sitting?” (fridge / arms), “Is the umbrella upside down?” (yes / no), “How many children are in the bed?” (2 / 1). Ayrıca “What game is this?” sorusu için cevabı “tennis” olmayan bir resim seçme arayüzü gösterilir.

Hoca fikri şöyle özetledi: bir soru A resmiyle verildiğinde cevabı bir şey, başka bir resimle verildiğinde başka bir şeydir. Bütün soruların bu formda olması sağlanırsa “What sport is” ile başlayanların %41’i tenis olmaz, “How many” ile başlayanların büyük kısmı 2 olmaz (00:38:50).

> **Ek bilgi (derste anlatılmadı):** Slayttaki arXiv 1612.00837, büyük olasılıkla Goyal ve arkadaşlarının “Making the V in VQA Matter” çalışmasıdır. Bu çalışma her soru için benzer ama farklı cevaplı tamamlayıcı resim çiftleri toplayarak VQA v2 veri kümesini oluşturmuştur.

## 8. Veri Kümesi Oluşturmanın Zorluğu ve Kısa Yol Öğrenme

### 8.1 Veri kümesi oluşturmak kolay değildir

Hocaya göre veri kümesi oluşturmak hakikaten zor bir iştir. Rastgele resimler çekip insanlara bunlarla ilgili soru sordurabilir, cevap verdirebilirsiniz. Ama insanların soru ve cevap hakkında ne üreteceğini yönetmek ve gözlemlemek kolay değildir. Bu süreç aslında işin çok önemli bir parçasıdır (00:34:46).

### 8.2 Kişiyi tanıyan model

Bazı çalışmalarda insanlara resimden metin ya da metinden resim ürettirildiğinde şu fark edilmiştir: üretilen çıktıyı A kişisinin mi, B kişisinin mi, C kişisinin mi ürettiği tanınabiliyorsa bu kötü bir şeydir. Bu durumda sistem problemin kendisini çözmek yerine doğrudan kişiyi tanımaya odaklanabilir ve bu gerçek dünyadaki performansı olumsuz etkiler (00:35:57).

Hoca bunu önceki haftanın mesajına bağladı: modeller bizim istediğimizi yaparlar, ama istediğimiz yoldan mı yaparlar? Orada soru işareti vardır (00:36:45). Makineler kısa yolları (shortcut learning) yakalamada çok iyidir. İstemediğimiz türden kısa yolların oluşup oluşmadığını gerçekten görmek gerekir. Bunlar çoğu zaman sistem gerçek zamanlı çalışmaya başladığında ya da belli durumlar oluştuğunda ortaya çıkar ve önceden kestirmek kolay olmayabilir (00:37:02).

> **Hocanın vurgusu:** Modeller bizim istediğimizi yaparlar, ama istediğimiz yoldan mı yaparlar? İstemediğimiz kısa yolların farkında olup ona göre hareket etmek gerekir.

### 8.3 Veri toplama tekdüze değildir

Slayttaki “Yeterli mi? Başka ne yapılabilir?” sorusu üzerine hoca, veri toplamanın tekdüze (uniform) bir süreç olmadığını söyledi. İnsanlar rastgele resim çekmez, rastgele yazı yazmaz; karşısındakinin ilgisini çekebilecek, bilgi içeren şeyler gösterir. Kimse içinde on yedi kedi olan

bir resim bulup “kaç kedi var?” diye sormaz. Dolayısıyla hayatın doğal akışına göre işin içinde bir yanlılık vardır (00:39:35).

Özellikle internetteki veri hayatın doğal akışıyla uyumlu değildir. Hocanın “çok klasik” dediği örnek: arama motoruna “güldü” yazın, sonra “öldürdü” yazın ve çıkan sonuç sayılarına bakın. Bu verilere bakınca insanların gülmekten çok öldürdüğü gibi bir sonuç çıkar (00:40:56). İnternet verisi yalnızca kolay erişilebildiği için çok kullanılır; bu noktada sıkıntılar ve yanlılıklar oluşabilir, hocaya göre zamanla bunlara da çözümler geliştirilecektir (00:41:22).

## 9. Karşıtsal Öğrenme ve SimCLR

### 9.1 Karşıtsal öğrenme

Hoca dersin başında önceden eğitilmiş modellerin ince ayar açısından iyi olduğunu söylemişti. Önceden eğitilmiş mimari oluşturmakta kullanılan yaklaşımlardan biri karşıtsal öğrenmedir (contrastive learning). Hoca bunu Türkçeye “karşıtsal öğrenme” diye çevirdiğini, tam uyup uyumadığından emin olmadığını söyledi (00:42:11).

Slayttaki maddeler (ekrandan, 00:42:20):

- 2 girdi için benzer ya da değil diyebilmek
- Çözüm: benzer olanlar için benzer, farklı olanlar için farklı çıktılar üretebilmek
- Eğitim kümesini oluşturmada eğiticili ve eğiticisiz yöntemler var
- Eğiticili: farklı 2 kedi resmi benzesin, 1 kedi ve 1 balık benzemesin

Bu öz denetimli (self-supervised) bir yaklaşımdır: “bu resim bir kedidir, bu resim bir köpektir, bu resmin içinde şu vardır” gibi insan eliyle bir etiketleme yapılmadan eğitim kümesi oluşturulur. Tek bir girdi için “içinde şu vardır” denmez; iki şey (iki resim ya da metin) verilir ve sistemden bunların birbirine benzediğini ya da benzemediğini, alakalı olup olmadığını söylemesi istenir (00:42:29). İnsan emeğine ihtiyaç olmadığı için hem etiketlemeden kaynaklanabilecek hatanın önüne geçilir hem de süreç çok ölçeklenebilir hale gelir: çok büyük veri kümeleri üretilebilir (00:50:50).

Hoca aradan ve bir ses sorunundan sonra slaytı yansıtmadığını, öğrencilerin slayta kendilerinin bakabileceğini ve 19. slaytta olduğunu söyledi (00:50:36). Bu yüzden SimCLR kısmı için ekran karesi yoktur; anlatım altyazıdan alınmıştır.

### 9.2 SimCLR

Hocaya göre karşıtsal öğrenmenin daha önce de birçok çalışması vardır, ama bu çalışmayla epey başarılı hale gelmiştir (00:51:15). Yapılan işlem bir resmin farklı parçalarını kullanmaktır: büyük bir resmin içinden rastgele seçilmiş kesitlerin (crop) birbirine benzer, farklı resimlerden seçilmiş kesitlerin ise birbirine benzemez olduğunu bulmak. Modelin tahmin etmeye çalıştığı, bu iki parçanın aynı resimden gelip gelmediğidir (00:51:39).

Kesilen parçalar üzerinde çeşitli dönüşümler de yapılır: renk değişimleri, çeşitli filtreler gibi. Hoca makalede anlamlı dönüşümün ne olacağını bulmak için epey zaman harcandığını ve her türlü dönüşümün o kadar işe yaramadığının söylendiğini aktardı. Amaç, iki parçanın birbirine benzediğini yakalamak için asıl önemli olan noktanın ne olduğunu bulmaktır (00:52:09).

Slayttaki ilk sütunda bir köpek resminden iki parça çıkar: birinde köpeğin kafası, diğerinde arka bacakları vardır. Şekilde parçalar iki farklı CNN’e veriliyormuş gibi görünür, ama bu CNN’ler ağırlık bakımından eştir (ikiz CNN’ler); aslında aynı ağın kopyasına verilirler. Çıkan vektörler bir MLP’den, yani iki tam bağlantılı katmandan (fully connected layer) geçer; hoca bunun doğrusal olmayan bir dönüşüm için olduğunu söyledi. Sonra MLP çıktıları arasında “bu buna benzer olmalı, bu bundan uzak olmalı” gibi geri beslemelerle sistem güncellenir (00:53:32).

İşlem bittiğinde elde sınıflandırma yapan bir şey yoktur; kabaca CNN’ler ve MLP kısmı vardır. Bu mimari sonraki görevler için önceden eğitilmiş model olarak kullanılır; örneğin sınıflandırma için ince ayar yapılır (00:54:45).

Metinler üzerinde de çok benzer çalışmalar vardır; bu durumda iki resim değil iki metin verilir (00:55:24).

**Ek bilgi (derste anlatılmadı):** SimCLR, büyük olasılıkla Chen ve arkadaşlarının 2020 tarihli “A Simple Framework for Contrastive Learning of Visual Representations” çalışmasıdır. Aynı resmin iki farklı artırılmış görünümünü bir kodlayıcı ve küçük bir izdüşüm başlığından (projection head) geçirip, aynı resimden gelenlerin benzerliğini artıran, farklı resimlerden gelenlerinkini azaltan bir karşıtsal kayıpla eğitilir.

### 9.3 Birden çok nesneli resimler

Resmin içinden parça kesip benzer saymak belli türden resimler için mantıklıdır. Resimde tek bir ana nesne varsa fikir fena değildir. Ama resimde bir sürü şey varsa, bir parçayla başka bir parçanın benzer olduğunu söylemek çok doğru olmayabilir (00:55:49).

Hoca yine de bunun işe yarayacağını söyledi. Bir resmin bir tarafında kedi, öbür tarafında köpek varsa kedinin kafasıyla köpeğin kuyruğunu birbirine yakın saymış olursunuz. Bir açıdan saçma gelir, çünkü içerik olarak benzemezler; kedi ile köpek, aynı kedinin kafasıyla kuyruğu gibi değildir. Ama dünyada bunlar birlikte bulunurlar. Bu birlikte bulunma (co-occurrence) bilgisi de ağa aktarılmış olur (00:56:13).

Sonraki slaytta otomatik kodlayıcılar (autoencoder) vardı; hoca bunlara bu derste girmedi (00:57:24).

## 10. Resimleri Anlamanın Sınırları

21. slaytta bazı resimler vardır ve hoca bunlarda ne olup bittiğini anlamanın bugünkü teknolojiyle hâlâ çok kolay olmadığını anlatmak istedi (00:57:41). Belli spesifik resimlerde ne olduğunu makineye söyletebilirsiniz; o resim ve ona benzeyen resimler için söyletebilirsiniz. Ama yeni bir resim geldiğinde, yani test zamanında, aynı başarı olacak mı? Hocaya göre bu hâlâ tamamlanmış bir alan değildir, soru işareti vardır (00:58:07).

Bu resimlerdeki konu mizahtır. Hocaya göre mizah zekânın en sağlam göstergelerinden biridir; çok iyi mizahçılar genelde çok zeki insanlardır, arada bir korelasyon vardır (00:58:48). Hoca şu spekülatif düşünceyi paylaştı: her şey belli temel şeylerin farklı karışımlarından oluşuyorsa, temel şeyler sabit ve karışımların permütasyonları sonsuzsa, yeni şeyler bulmak yalnızca yeni birleşimlerden ibaretse, makinenin yeni esprileri anlama ihtimali vardır. Ama bu, dümdüz, tekdüze işleyerek olmaz gibi görünür: şu anki mimarilerde girdi verilir, katman katman ilerler ve iş biter; hiçbir şey diğerinden daha uzun ya da kısa sürmez. Hocaya göre belli noktalarda döngüsel bir yapıya ihtiyaç olabilir. Bunların hepsinin spekülatif ifadeler olduğunu belirtti (00:59:20).

**Soru:** Bir öğrenci aynı soruyu ChatGPT’ye ve LLaMA ağırlıklarını kullanan Vicuna’ya sorduğunu, farklı ağırlık ve yapılar olmasına rağmen (örneğin bir tanım istediğinde) benzer cevaplar aldığını söyledi (01:00:44). **Cevap:** Hoca, benzer veri kümeleriyle eğitilmiş olmalarının yanında, insanların makinelere soracağı soruların çeşitliliğinin mevcut veri kümeleriyle büyük ölçüde açıklanabiliyor gibi göründüğünü söyledi. Yepyeni şeyler karşısında nasıl kabiliyetler gösterecekleri ve ilerideki sürümlerin neler yapacağı merakla görülecektir (01:01:35).

## 11. Ödev 3 Sunumları: Üç Anahtar Kelimeden Cümle Üretimi

### 11.1 Görev ve değerlendirme

Dersin ikinci yarısında üçüncü ödevin sunumları yapıldı. Hoca her sunuma yaklaşık 5 dakika verdi ve ödevin tanımı herkes için aynı olduğundan “ne yaptığınızı değil, nasıl yaptığınızı ve nasıl sonuçlar aldığınızı” anlatmalarını, yöntem ve bulgulara odaklanmalarını istedi (01:02:42). Görev, verilen üç anahtar kelimeyi (keyword) içeren anlamlı bir cümle üretmektir. Hoca ilk sunumdan sonra beklentisini şöyle hatırlattı: en az 50 kelime üçlüsü için yöntemler sayısal olarak, insan yargısını da içerecek şekilde karşılaştırılmalı ve sonunda “bu 50 örneğin 35’inde FLAN-T5, 15’inde LSTM daha iyi çalıştı” gibi bir sonuç verilmeli. Yalnızca üç örnek cümle göstermek yeterli değildir (01:19:32).

Aşağıda sunumların akademik içeriği, sunan kişiler anılmadan özetlenmiştir.

### 11.2 FLAN-T5 ve LSTM

Bir öğrenci problemi metin üretimi (text generation) görevi olarak ele aldı ve hem RNN/LSTM gibi daha temel yaklaşımları hem de Transformer tabanlı modelleri denedi (01:15:22). Transformer tarafında çok dilli (multilingual) olduğu için FLAN-T5 mimarisine yoğunlaştı. İnce ayar için hedef değişken gerekiyordu: baktığı örneklerde model anahtar kelimelerden cümleye eğitiliyordu. Veri kümesinde anahtar kelimeler hazır olmadığı için her paragraftan BERT tabanlı bir anahtar kelime çıkarıcıyla (Türkçe anahtar ifade çıkarma üzerine bir çalışmada en iyi bulunan model) üçer anahtar kelime çıkardı ve FLAN-T5'i bunlarla ince ayarladı; aynı yapıda bir LSTM ağı da eğitti (01:17:14).

Gözlemi, FLAN-T5'in LSTM'e göre daha anlamlı cümleler ürettiğiydi. Slayttaki tablo (ekrandan, 01:20:00):

| örnek kelime üçlüsü | FLAN-T5 | LSTM |
|---|---|---|
| piyasa, dolar, beklenti<br>ekonomi, makro, yükseliş<br>sanayici, kurumsal, görüş | piyasada dolar beklentiyi aştı<br>makro ekonomi yükselişte<br>sanayici kurumsal güven görüştü | piyasa dolar beklentisi<br>ekonomi makro yükselişte<br>sanayici görüşü |

Öğrenci epoch sayısı gibi ayarları artırarak FLAN-T5'i daha ileri taşıyabileceğini, GPT-Neo ile yapılan çalışmaları da deneyebileceğini söyledi (01:18:44).

**Soru:** Hoca üç örnekten daha geniş kapsamlı bir karşılaştırma tablosu olup olmadığını sordu. **Cevap:** Öğrenci kelime üçlülerinin tümü için sonuçları bir dosyaya kaydettiğini, büyük çoğunlukta (30'un üzerinde) anlamlı cümleleri FLAN-T5'ten aldığını, ROUGE metriğini de denemek istediğini ama uygulayamadığını söyledi (01:19:47).

### 11.3 Markov zinciri, kısıtlı ışın araması ve üretici modeller

İkinci sunumda yaklaşık 42 bin haberden oluşan bir derlem kullanıldı (01:22:03). Sunum büyük ölçüde İngilizce yapıldı ve altyazının bir kısmı bozuktur.

- **Markov modeli (3-gram ve 5-gram):** Trigramlarla bir Markov geçiş matrisi kurulur. Başlangıç durumundan itibaren kısıt kelimelerinden (constraint words) birini içeren sonraki durum aranır. Bulunamazsa geçiş matrisinden rastgele seçilir; bulunursa o durum seçilir. Bütün kısıt kelimeleri bulunduktan sonra üretim durmaz; cümle sonu tokenı bulunana kadar devam eder. Sonra özel tokenlar silinir, yeni cümleler büyük harfle başlatılır, normalizasyon yapılır. İkinci deneyde aynı süreç 5-gramlarla yapıldı (01:24:40).
- **Kısıtlı ışın araması (constrained beam search):** Her ışında en olası üç durum üretilirken istenen kelime de (örneğin ilk aşamada "is", ikinci aşamada "fast") zorla eklenir; böylece kelimenin çıktıda görünmesi sağlanır (01:26:38).
- **Sorun:** Türkçe bir örnekte başlangıç metni verilip "hava" kelimesi zorlandığında cümle kabul edilebilir çıktı, ama zorlanan kelime cümlenin sonuna yerleşti. Sıcaklık (temperature), top-k ve top-p parametreleri değiştirildiği halde zorlanan kelimeler yine sonda kaldı; bu istenen bir şey değildir (01:27:33).
- **Üretici model eğitimi:** Girdisinde anahtar kelimeler, çıktısında onları içeren cümle olan bir model eğitmek için açık kaynaklı Keytotext çerçevesi kullanıldı (01:28:02).

Hoca sonunda GPT-2 ve mT5'te anlamlı bir şey oluşmadığı için karşılaştırmanın Markov'un 3-gram ve 5-gram sürümleri üzerinden yapıldığını özetledi (01:31:48).

### 11.4 GPT, Word2Vec ile yönlendirme ve mT5

Bir başka öğrenci birkaç yöntemi deneyip bu problem için en iyisini aradı (01:32:29).

- **Rastgele başlangıç ve top-k:** Üretici bir modelde üç kelimeden biri rastgele başlangıç olarak seçilir, sonra en olası top-k token içinde diğer kelimelerden biri var mı diye bakılır. $k$ küçükken birbirine yakın kelimelerde bile iyi sonuç çıkmadı; $k$ çok büyükken paragraflarca, anlamlı bitmeyen ve üç kelimeyi barındırmayan çıktılar oluştu. İki farklı GPT modeli kullanıldı (01:32:44).

- **Word2Vec ile yönlendirme:** Seçilecek token, hedef kelimelerin gömmelerine en yakın olan top-k aday olarak seçildi (Wikipedia üzerinde, geniş pencereyle eğitilmiş bir Word2Vec modeli). Sonuç başarısızdı. Nedenleri: dil modelinin tokenizer'ı (SentencePiece gibi) kelimeleri parçalara böler, Word2Vec'te bu parçaların vektörü yoktur; iki anlamsız token birleşince anlamlı bir şey çıkabilir ama seçilemez; ayrıca sürekli durak kelimeleri (stop word) seçilir (01:33:57).
- **mT5 ince ayarı:** Haber metinleri çok sıkı kurallarla filtrelenerek 92 bin küsur cümleye inildi; durak kelime olmayan anahtar kelimeler seçildi. Model girdisi "cümle üret: " biçiminde, çıktısı cümledir; mT5'in small sürümü eğitildi (01:36:08).
- **Kısıtlı ışın araması:** İstenen tokenlar ışın aramasında eklenmiş gibi varsayılarak diğer olasılıklarla karşılaştırılır. İki GPT modeli arasında karşılaştırıldıktan sonra daha iyi olan seçildi (01:37:10).

Değerlendirmede 54 örnek kullanıldı. Kısıtlı ışın aramasının ürettiği cümleler daha uzun olsa da bütünsel olarak daha iyi anlam taşıdı. Anahtar kelimelerin ortalama %42,76'sı üretilen cümlelerde yer aldı: her örnekte üç anahtar kelimenin ne kadarının kullanıldığı oranlanıp ortalaması alındı. mT5'te bu oran daha yüksekti, ama anlamsız cümlelere puan verilmediği için anlamlılıkta kısıtlı ışın araması tercih edildi (01:37:45). Slayttaki örnekler (ekrandan, 01:40:00):

| Anahtar kelimeler | Kısıtlı ışın araması | mT5 |
|---|---|---|
| geceyarısı, sessizlik, ay | ay geceyarısı belirgindi sessizlik hakimdi | geceyarısı'sayın sessizlik' olarak biliniyor. |
| renkli, balon, gökyüzü | balon gökyüzüne doğru uçtu. renkli anlar kameraya yansıdı | balon gökyüzü renkli. |
| harika, manzara, doğa | manzara. Tags: … | - doğa harika bir manzara. |
| yaralı, kuş, kanat | kanatlı olan kuşun yaralı kanadı | yaralı kanat, yaralı yaralı olarak kabul etti. |

Üçüncü örnekte kısıtlı ışın araması "manzara"dan sonra başka bir dilde metin üretmeye başladı; öğrenci bunu modelin çok dilli olmasına bağladı (01:39:56). Bir n-gram modeli de eğitildi ama sonuçlar çok tekrarlıydı ve konu zaman zaman sapıyordu (01:40:05).

**Hocanın eklemesi:** Word2Vec'te herhangi bir kelimenin vektörüne erişemeyebiliriz, çünkü kelime sözlükte olmayabilir. Bunun çözümü FastText'tedir: FastText alt kelime (subword) parçalarını kullanır, tıpkı tokenizer'larda olduğu gibi, ve herhangi bir kelime için bir vektör üretebilir. Ne kadar kaliteli ürettiği ayrı bir konudur (01:40:52).

> **Ek bilgi (derste anlatılmadı):** FastText bir kelimeyi karakter n-gramlarının toplamı olarak temsil eder. Bu yüzden eğitimde hiç görülmemiş bir kelimenin vektörü de, içerdiği karakter n-gramlarının vektörlerinden kurulabilir.

### 11.5 BERT ile çoklu maske ve LSTM

Bir öğrenci üç ana yöntem kullandı ve 50 kelime üçlüsü için ürettiği cümleleri bir skorla değil insan yargısıyla karşılaştırdı (01:42:14). Word2Vec ile yaptığı deneme kötü sonuç verdiği için sunuma eklenmedi.

- **BERT çoklu maske:** Girdi "<1. kelime> [MASK] <2. kelime> [MASK] <3. kelime> [MASK]" biçimindedir; BERT maskeleri tahmin eder. Örnekler: "oyun, çocuk, herkes" → "oyun sırasında çocuk olan herkes katılabilir"; "futbol, para, çok" → "futbol ile para kazanmak çok önemli" (01:43:32). Her anahtar kelimeden sonra iki maske konan sürümde "gözlük takma ile çocuk zarar görmüş olabilir" gibi daha anlamlı cümleler çıktı (01:45:06).
- **LSTM ile son kelime tahmini:** Bir anahtar kelime verilir, bir ya da iki kelime tahmin ettirilir, sonra ikinci anahtar kelime eklenip yine tahmin ettirilir, sonra üçüncü. Örnek: "oyun, çocuk, herkes" → "oyun bir çocuk tarafından herkes bir". Kısa cümlelerde LSTM anlamsız üretti; LSTM bazen bir eki ayrı bir kelime gibi üretti (01:44:42).
- **Derlem etkisi:** İlk LSTM'in eğitildiği derlemdeki cümleler çok uzun olduğu için 7-8 kelimelik bir cümle üretemiyordu. Cümleleri daha kısa olan bir yorum derlemiyle eğitilince daha anlamlı sonuçlar çıktı: "ürün, kargo, çok" → "ürün güzel ama kargo çok geç" (01:47:05).

- **İsimler-fiiller derlemi:** Hocanın paylaştığı isimler ve fiiller derlemindeki açıklama sütunları kullanılarak, her kelimenin karşılığı birleştirilip cümle kurulmaya çalışıldı ("motor, anahtar, bulmak" için). Öğrenci bunu tam bir yöntem değil, bir olasılık olarak gösterdi (01:47:46).

Sonuç: BERT, LSTM'e göre daha anlamlı cümleler üretti; LSTM daha uygun bir derlemle eğitilince iyileşti ama BERT düzeyine ulaşmadı (01:48:36). Hoca LSTM'in eğitildiği derlemin büyüklüğünün de çok kritik olduğunu, çok büyük bir derlemle eğitilirse farklılıklar oluşabileceğini ekledi (01:49:01).

### 11.6 Markov (1-gram) ve LSTM ile film yorumları

Bir öğrenci IMDb film yorumları veri kümesini kullandı (01:50:03). Ön işleme: gereksiz karakter, sembol ve HTML'nin temizlenmesi, küçük harfe çevirme, tokenizasyon, durak kelimelerin çıkarılması (01:50:54).

- **Markov zinciri:** Veri toplama, tokenizasyon, geçiş matrisinin kurulması ve geçiş olasılıklarının hesaplanması adımlarından oluşur. Bir tokendan diğerine geçiş olasılığı, o geçişin frekansının mevcut tokenın toplam geçme sayısına bölümüdür. Uygulamada kelime, sonraki kelime ve frekans sütunlu bir tablo kurulup ardışık çiftlere göre gruplanır, kelimeyi indeks ve sonraki kelimeyi sütun yapan bir pivot tabloyla olasılıklar hesaplanır. Üretim rastgele seçilen bir tokenla başlar; cümle sonu kelimesine gelinince ve uzunluk sınırı aşılınca durur (01:52:24).
- **LSTM:** Her kelime one-hot ya da Word2Vec/GloVe gibi gömmelerle temsil edilir, LSTM diziyi kelime kelime işler, çıktı softmax ile sözlük üzerinde bir olasılık dağılımına çevrilir ve buradan örnekleme yapılır. Eğitim dizileri kaydırmalı pencereyle oluşturulur (01:55:03).

Örnek: "scary, horror, terrifying" için "The movie is scary and filled with terrifying moments" gibi cümleler üretildi (01:58:14). Öğrencinin sonucu: LSTM tutarlılık ve karmaşık dil örüntülerini anlamada genelde daha iyidir; Markov modeli sınırlı veride ya da basitlik ve hızın önemli olduğu durumlarda yine etkili olabilir (01:59:07).

**Soru:** Hoca LSTM'in Markov'dan hep daha iyi olup olmadığını, sayısal bir şey söylenip söylenemeyeceğini sordu. **Cevap:** Öğrenci kısa cümlelerde Markov'un, uzun cümlelerde LSTM'in daha iyi çıktığını, ama bunu saymadığını söyledi (02:00:05).

### 11.7 RNN, LSTM ve GPT ile devam ettirme

Bir öğrenci Türkçe film yorumları veri kümesiyle RNN, LSTM ve GPT modellerini, İngilizce derlem için ayrıca Keytotext modelini denedi (02:00:51). Cümle uzunluğu yedi kelimeye ayarlandı; örneğin "altyazı seçenekleri" ile başlayan girdiler devam ettirildi. GPT modeli Hugging Face'ten alındı (02:01:42).

**Soru:** Hoca bunun ödevin tam istediği şey olmadığını, üç kelimeyi verip yalnızca devamını getirmek olduğunu söyledi; öğrenci doğruladı (02:04:22).

### 11.8 BERT temsil benzerliği ve çift yönlü Markov zinciri

Bir öğrenci yaklaşık 60 MB'lık Türkçe cümle derleminden bir sözlük oluşturdu ve kelime üçlülerini hocanın isimler-fiiller derlemindeki isimlerden, sözlükte 10'dan az geçenleri eleyerek rastgele seçti (02:05:10).

- **BERT ile benzerlik:** Derlemden 15-25 kelimelik, üçlüdeki hiçbir kelimeyi içermeyen bir cümle rastgele seçilir ve BERTurk temsili alınır. Üçlüdeki her kelime, cümledeki her kelimenin yerine ayrı ayrı konarak yeni cümlenin BERT temsili alınır; orijinal cümlenin temsiline en yakın sonucu veren konuma yerleştirilir. İki kelime aynı konumu gösterirse ikincisi ikinci en benzer konuma konur (02:05:47).
- **Çift yönlü Markov zinciri:** Kelime tabanlı 2-gram Markov matrisi hem ileri hem geri yönde, cümle başı ve sonu özel tokenlarıyla hazırlandı. Birinci kelime için geri yönde en fazla yedi kelimelik ve solda cümle başı işareti olan bir tahmin alınıp cümle başı yapılır. Sonra birinci kelime için ileri, ikinci kelime için geri yönde yüzer tahmin alınır ve bunların ortak bir kelimede kesişip kesişmediğine bakılır; kesişince iki parça bu kelime üzerinden birleştirilir. Aynı işlem üçüncü kelimeye kadar yapılır ve son olarak cümle sonu işareti

içeren ileri bir tahminle cümle bitirilir. 50 örneğin 46'sında çalıştı; kalan 4'ünde yedi kelimelik pencere yetmedi, 12'ye uzatılınca bunlar da üretildi (02:06:54).

İki yöntem de başarılı cümle vermedi. Öğrencinin gelecek çalışma önerileri: BERT yönteminde temel cümlede yakın anlamlı kelime varsa başarı artar (ama bu istenen yön değildir); Markov yönteminde pencere genişletilip birleştirme tek kelime yerine yan yana gelen üç ya da daha fazla ortak kelime üzerinden yapılabilir (02:09:05).

### 11.9 5-gram ile kelime yerleştirme

Bir öğrenci hocanın paylaştığı haber derlemindeki 12.663 metin dosyasını tek metin gibi birleştirip 5-gram bir model kurdu (02:11:03). Bir anahtar kelimeyle başlayıp sonraki olası kelimeler arasında diğer anahtar kelimeler varsa onları yerleştirmek, yoksa rastgele bir kelime seçmek istedi. Kelimeler rastgele seçildiği ve çoğu yan yana gelen kelimeler olmadığı için başarılı olmadı; GPT ile denemelerde de aynı kelimenin tekrarıyla karşılaştı (02:12:15).

### 11.10 Anahtar kelime seçimi ve küçük dil modelleri

Bir öğrenci 9 sınıflı bir haber veri kümesinden her sınıftan 200 haber alıp cümlelere ayırarak 26 bin örnek üretti (02:15:30). Lematizasyon, gövdeleme (stemming), durak kelime ve ek atma gibi temizleme yöntemlerini denedi; kodlayıcı-çözücü modellerde bunların zarar verdiği sonucuna varıp yalnız boşlukla tokenizasyon ve küçük harfe çevirmeyi kullandı (02:15:44).

Üç kelimeyi seçmek için birçok yöntem denedi: rastgele, ilk üç kelime, en sık üç kelime, önceden eğitilmiş BERT ile token önemi, TF-IDF ile token önemi, Word2Vec'te birbirine en çok benzeyen üç kelime, sözcük türü etiketleme (part of speech tagging) ile iki isim ve bir fiil, CountVectorizer ile frekans ve TextRank. Son veri kümesinde sözcük türü etiketlemeyle seçilen kelimeler kullanıldı (02:16:32).

Cümle üretimi için:

- **Kodlayıcı-çözücü:** Sözlük boyutunda girdi, bir LSTM katmanıyla kodlama, bir LSTM katmanıyla çözme ve sonda tam bağlantılı bir sınıflandırıcı (02:17:44).
- **Dil modeli:** Tek LSTM katmanlı bir model; ikinci sürümde birkaç katman (dropout, GELU) eklendi ve tokenlar birleştirildi. Girdi "başlangıç tokenı, üç kelime, ayırıcı token, cümle" biçiminde; çıktı aynı dizinin bir kaydırılmış hali (baştaki start silinip sona end eklenmiş). Böylece model bir sonraki tokenı öğrenir; bu sürüm bir öncekinden biraz daha iyi sonuç verdi (02:19:01).
- **T5 ince ayarı:** Çoğu katman dondurularak denendi, başarılı olmadı (02:19:39).
- **Word2Vec yolu fikri:** Seçilen kelimeleri ikili ikili alıp vektör uzayında aralarındaki yoldan kelime seçmek; seçilen kelimelerin cümleye nasıl dönüştürüleceği çözülemedi (02:20:00).

Modeller ya ilk üç kelimeyi yazıp bitirdi ya da anlamsız bir kelimeyi tekrarlayarak devam etti. Tokenları birleştiren dil modeli ilk dil modeline göre daha iyi tahminler yaptı; kodlayıcı-çözücünün tahminleri neredeyse hiç anlamlı değildi (02:21:58).

**Hocanın önerisi:** Model kelime tabanlıydı (yaklaşık 42 bin kelimelik bir sözlük). Hoca kelimeleri doğrudan kullanmak yerine BERT tokenizer'ı gibi alt kelime tabanlı bir şeyden geçirip bunları token olarak kullanmanın büyük olasılıkla daha başarılı olacağını söyledi. Veri kümesi zaten küçüktür; saf kelime tabanlı bir yaklaşım, özellikle Türkçe için çok fazla seyreklik yaratır. Kelimeler ek ve köklere dönüştürülseydi daha iyi sonuç alınabilirdi (02:22:43).

### 11.11 Markov zinciri ve GPT-2 ile roman metni

Bir öğrenci hocanın paylaştığı romanlar derleminden bir romanın yaklaşık 30 bin kelimelik kısmını kullandı (02:23:58). İlk model iki kelimelik n-gramlı bir Markov zinciriydi: sonraki kelimeler arasında istenen kelimelerden biri varsa o konur, yoksa istenen kelimeler olası kelimeler arasına eklenir, cümle bitene kadar devam edilir. Romandaki özel isimler nedeniyle çok anlamlı cümleler çıkmadı (02:24:26).

İkinci model aynı veride eğitilen Türkçe GPT-2'ydi; 30 bin kelime seçilmesinin nedeni GPT-2'yi eğitebilmekti. Öğrenci GPT-2'nin ChatGPT gibi talimatla "bu üç kelimeyle cümle üret" görevini

yapabileceğini düşündüğünü, ama öyle olmadığını, modelin yalnızca verilen kelimelerle başlayan cümleler ürettiğini söyledi. Anlamlılık açısından GPT-2 daha iyiydi, ama anahtar kelimeleri kullanma görevini yerine getiremedi (02:25:22).

### 11.12 LSTM ve GPT-2 ile haber verisi

Bir öğrenci birinci ödevdeki haber sınıflandırma verisini sınıfları olmadan ham veri olarak kullandı (02:27:44). İlk mimari kelime indeksiyle vektörleştirme (sözlükteki her kelimeye bir token numarası) ve üzerine bir LSTM'di; ikincisi ince ayarlanmış bir Türkçe GPT-2'ydi (Hugging Face'teki açık kaynaklı modellerden biri) (02:28:02). LSTM sürekli tekrar etti; sözlükte olmayan kelimeler bilinmeyen (unknown) token oldu. Daha büyük veriyle sözlük büyür, ama kelime vektörü boyutu da büyüdüğü için model boyutu ve çıkarım süresi artar (02:34:10). GPT-2 en azından ilk cümleye kadar anlamlıydı, ama sonra haber ajansı adları, "bilgilerinizi almak için buraya tıklayın" gibi haber verisinden gelen alakasız ifadeler ekledi (02:35:13).

**Soru:** Hoca LSTM'in eğitimde girdisinin ve çıktısının ne olduğunu, yani nasıl eğitildiğini sordu. Bir sonraki kelimeyi tahmin etmek bir şeydir, cümlenin ilk üç kelimesini verip tamamını tahmin ettirmek başka bir şeydir (02:30:41). **Cevap:** Öğrenci önce test zamanını anlattı; hoca test zamanını değil eğitim sürecini sorduğunu belirtti. Sonunda eğitimin, bir cümlenin ilk birkaç kelimesi verilip o cümlenin tamamının üretilmesi biçiminde olduğu anlaşıldı (02:33:36).

**Soru:** Hoca LSTM ve GPT sonuçlarının karşılaştırılıp karşılaştırılmadığını, insan yargısıyla nasıl değerlendirileceğini sordu. **Cevap:** Öğrenci sayısal karşılaştırma yapmadığını, insan yargısıyla GPT-2'nin daha iyi olduğunu, kelime tekrarının daha az ve ilk 8-10 kelimenin genelde mantıklı olduğunu söyledi (02:36:04). Hoca GPT çıktılarının da hep o üç kelimeyle başladığını, oysa kelimelerin böyle sıralı olmamasının ödevin amacı olduğunu hatırlattı (02:37:32).

### 11.13 N-gram Markov ve BERT maskeleme ile edebi metin

Bir öğrenci hocanın paylaştığı veri setlerinden bir romanın metnini kullandı (02:39:52). Kelime kelime Markov zinciri (unigram) başarılı olmadı; gizli Markov modeli (Hidden Markov Model) kodlamada hata verdi. Sonra n-gramlı bir Markov zinciri kuruldu. Metinden rastgele 50 üçlü seçildi; üçünden biriyle başlanıp önce o kelimenin bir trigramı var mı diye bakılır, varsa sonraki kelime oradan seçilir; trigram yoksa bigrama bakılır; o da yoksa veri setinden rastgele bir kelime seçilir. Cümle en fazla 50 kelimeyle sınırlandı; üç kelime bulunduktan sonra nokta, soru işareti ya da ünlem gibi bir bitiş işaretine gelince cümle biter (02:41:10).

Slayttaki sonuç: 2-gram ve 3-gram kullanılmış; üretilen 50 cümlenin 29'u anlamlı sayılabilecekken 21'i anlamsız olmuş; başarı oranı %58 hesaplanmış (ekrandan, 02:40:00). Kitabın ağdalı, edebi dili olduğu için ufak tefek saçmalıklar göze batmadı (02:43:01).

İkinci yöntem BERT ile maskelemeydi: bütün 50 cümlede aynı maskeleme düzeni kullanıldı ve 24 cümle anlamlı bulundu, başarı oranı %48 (02:44:42). "teker teker" gibi ikilemelerde BERT aynı kelimeyi üç dört kez art arda kullandı; bunu önlemek için art arda en fazla iki maske konuldu (02:45:16).

Öğrencinin karşılaştırması: n-gram daha anlamlı cümleler ve daha yüksek başarı verdi (%58'e karşı %48), ama tüm kelimeleri kullanmak için cümleler çok uzayabilir ve uzunluk sınırı konunca başarısız da olabilir; sınır konmazsa çok uzun süre çalışabilir. BERT'te maskeleme yapıldığı için istenen uzunlukta cümle doğrudan üretilir. Seçilen kelimelerin isim ya da fiil olması n-gramın başarısını artırır; edat ve bağlaç gibi kelimeler düşürür. BERT'in başarısı ise seçilen kelimeden çok etkilenmez, çünkü boşlukları doğrudan doldurur (02:46:45).

### 11.14 Bigram LSTM ve BERT maskeleme ile film yorumları

Bir öğrenci film yorumlarından oluşan bir veri setinde sayıları ve noktalama işaretlerini temizleyip her şeyi küçük harfe çevirdi; noktalama işaretleri üretilen cümlelerde anlamsız yerlere geliyordu (02:49:23). Birinci yöntemde bigramlar oluşturulup LSTM'e verildi; ikinci yöntemde Türkçe BERT ile "[MASK] <1. kelime> [MASK] <2. kelime> [MASK] <3. kelime> [MASK]" yapısında cümle üretildi (02:50:00). İkililer anlamlı olsa da LSTM'in cümlelerinin bütününde anlam yoktu; BERT her durumda daha anlamlı cümleler üretti. Ayrıca önceden eğitilmiş Türkçe

dil modeli olduğu için verilen kelimeleri kendisi çekimledi ("film" → "filmin") (02:51:04). Öğrencinin önerileri: trigram, farklı kelime vektörleri ve daha büyük ya da daha temiz bir veri seti (02:53:11).

**Soru:** Hoca BERT maskelemede eğitim süreci olmadığını, eğitilmiş BERT'in üzerine yapıldığını doğruladı. **Cevap:** Öğrenci ince ayar yapmadığını, yapılsa veri kümesine daha çok yaklaşabileceğini söyledi (02:54:03).

### 11.15 BERT maske ve T5-small ince ayarı

Bir öğrencinin birinci yönteminde Türkçe BERT ile üç kelimelik isteme rastgele bir konuma maske tokenı eklenip BERT'e tahmin ettirildi. İkinci yöntemde Keytotext kütüphanesinin eğitim aracıyla T5-small ince ayarlandı: Türkçe bir veri setinin yaklaşık 14 bin örneklik test kısmı kullanıldı, 10 epoch çalıştırıldı; her metnin anahtar kelimeleri Keytotext'in KeyBERT bileşeniyle (anahtar kelime sayısında sınır olmadan) çıkarıldı (02:55:08). BERT yöntemi daha iyi sonuç verdi (02:56:20).

**Soru:** Hoca İngilizce eğitilmiş T5'in Türkçe veriyle ince ayarlanıp ayarlanmadığını sordu; öğrenci doğruladı. **Cevap:** Hoca bunun çok anlamlı olmadığını, büyük olasılıkla sıfırdan eğitmekle bir fark yaratmadığını ve o büyüklükte bir model için veri kümesinin çok küçük kaldığını söyledi. İngilizce bir modeli Türkçe veriyle ince ayarlamanın ne sağlayacağı belirsizdir (02:57:00).

### 11.16 Kurallı cümle türetme ve Markov n-gram

Son sunumda hocanın paylaştığı haber derleminin 12.663 dosyası tek veri setinde birleştirildi; cümlelere ayırma, noktalama ve sık geçen ama tek başına anlamsız kelimelerin temizlenmesi, küçük harfe çevirme yapıldı (02:58:32).

- **Kurallı cümle türetme:** Türkçe cümle yapısı (özne, nesne, fiil) temel alınır; kelimelerin sözcük türü etiketlenir ve girilen kelimeler bu yapıya göre yerleştirilir (03:00:22).
- **Markov n-gram:** $n = 1$ ve $n = 2$ ile iyi sonuç alınmadı, $n = 3$ ile sonuç elde edildi (03:02:10).
- **LSTM:** Eğitimde çıktının (Y) ne olacağı oturtulamadığı için sonuç alınamadı; öğrenci bunun hocanın önceki sunumda sorduğu soruyla aynı sorun olduğunu söyledi (03:00:50).

Örnek: "Cumhurbaşkanı, son, durumun" için kurallı yöntem yarım kalmış bir cümle üretirken, 3-gram Markov "Cumhurbaşkanı son durumun çok iyi bir dünyayı tasvir eden fotoğrafları paylaşmasını istiyor" cümlesini üretti. Markov cümleyi bir fiille ve noktayla bitirerek Türkçe yapıya uygun tamamladığı için daha başarılı bulundu (03:04:55).

## Örnek Sorular

**Soru 1** (ekrandan, 00:20:00): Show and Tell'de optimizasyon $\theta^\star = \arg\max_\theta \sum_{(I,S)} \log p(S \mid I; \theta)$ ve $\log p(S \mid I) = \sum_{t=0}^{N} \log p(S_t \mid I, S_0, \ldots, S_{t-1})$ olarak verilir. $\theta$ neyi kapsar ve ikinci ifade modelin nasıl bir model olduğunu gösterir?

**Cevap (derste verildi):** $\theta$, CNN (kodlayıcı) ile RNN/LSTM'in (çözücü) hepsinin birden parametreleridir; bütün (resim, tanım) ikilileri için doğru tanımı en olası kılan parametreler aranır. İkinci ifade, tanımın tek seferde değil adım adım üretildiğini gösterir: resim ve o ana kadarki kelimeler verildiğinde bir sonraki kelime tahmin edilir. Bu tam bir otoregresif modeldir (next token prediction).

**Soru 2** (derste sözlü, 00:24:06): Çok büyük bir modeli çok az veriyle eğitirsek ne olur, ne yapmak mantıklıdır?

**Cevap (derste verildi):** Model eğitilebilir, ama girdideki küçük bir değişiklik çıktıda büyük bir değişikliğe yol açabilir; bu, modelin varyansının yüksek olduğu anlamına gelir. Veri arttıkça varyans azalır. Çok veri yoksa önceden eğitilmiş bir model kullanmak mantıklıdır; daha küçük bir model kullanmak ise daha az soyut örüntülerle yetinmek demektir.

**Soru 3** (ekrandan, 00:27:10): Tanım üretiminde ışın araması (beam search) nasıl çalışır ve Show and Tell'de en olası cümleler hakkında ne gözlenmiştir?

**Cevap (derste verildi):** O ana kadarki en olası 20 cümle tutulur ve $S = \arg\max_{S'} p(S' \mid I)$ seçilir. Üretilen 20 cümleden en olası olanı seçildiğinde %80'i eğitim kümesindeki cümlelerdendir; bu, modelin çok çeşitli cümle üretemediğini ve yeni resimlere uyum yeteneğinin sorgulanması gerektiğini gösterir.

**Soru 4** (ekrandan, 00:33:30): Görsel soru cevaplamada aynı eğitilmiş modele resim yerine boş bir resim verilip cevaplara bakılıyor. Bu deneyden ne beklenir?

**Cevap (derste verildi):** Model resme gerçekten bakıyorsa resmi gördüğü ve görmediği durumdaki cevapların farklı olması beklenir. Cevaplar aynı kalıyorsa model resimden değil soru kalıplarından öğreniyordur. Slayttaki örneklerde bazı sorularda cevaplar farklıdır, bazılarında aynıdır.

**Soru 5** (ekrandan, 00:40:00): VQA veri kümesinde "Bu resimde X var mı?" sorularının %81'inin cevabı "evet"tir. Resme hiç bakmadan bu tür sorulara hep "evet" diyen bir sistem bu sorularda ne kadar başarılı olur ve bu neden sorundur?

**Cevap (derste verildi):** Yaklaşık %81 başarılı olur. Bu bir yanlılıktır: insanlar böyle bir soruyu genelde ancak resimde o nesne varsa sorar. Makineler bu düzenliliği çok kolay yakalar ve başarıları abartılır. Çözüm olarak aynı soru için farklı resimlerde farklı cevaplar olan dengeli bir veri kümesi önerilmiştir.

**Soru 6** (ekrandan, 00:38:55): Veri kümesi aynı sorunun farklı resimler için farklı cevapları olacak biçimde güncelleniyor. "Yeterli mi? Başka ne yapılabilir?"

**Cevap (derste kısmen verildi):** Hoca veri toplamanın doğası gereği tekdüze olmadığını vurguladı: insanlar rastgele resim çekmez, rastgele yazmaz, ilgi çekici olanı paylaşır. Bu yüzden başka "ortaklıklar" (yanlılıklar) da olabilir; örneğin internette "öldürdü" için çıkan sonuçların "güldü"den fazla olması gibi. Ayrıca soru-cevapları kimin ürettiğinin tanınabilmesi de istenmeyen bir kısa yoldur. Hoca slayttaki soruya kesin bir yöntem listesi vermedi.

**Soru 7** (derste sözlü, 00:55:49): SimCLR'da aynı resimden kesilen parçalar benzer kabul edilir. Resimde bir kedi ve bir köpek varsa bu yaklaşım ne öğretir?

**Cevap (derste verildi):** Kedinin kafasıyla köpeğin kuyruğu benzer sayılır. İçerik olarak bu doğru değildir, ama bu iki şeyin dünyada birlikte bulunduğu (co-occurrence) bilgisi ağa aktarılır. Tek ana nesneli resimlerde ise fikir doğrudan "aynı nesnenin parçaları benzer" bilgisini öğretir.

**Soru 8** (derste sözlü, 01:40:52): Word2Vec ile bir dil modelinin ürettiği tokenların gömmelerini almak istediğimizde birçok token için vektör yoktur. Ne yapılabilir?

**Cevap (derste verildi):** FastText kullanılabilir. FastText, tokenizer'larda olduğu gibi alt kelime (subword) parçalarını kullanır ve herhangi bir kelime için bir vektör üretebilir; kalitesi ayrı bir konudur.

**Soru 9** (derste sözlü, 02:22:43): Küçük bir Türkçe veri kümesinde kelime tabanlı (yaklaşık 42 bin kelimelik sözlük) bir dil modeli kötü sonuç veriyor. Hocanın önerisi nedir?

**Cevap (derste verildi):** Kelimeleri doğrudan kullanmak yerine BERT tokenizer'ı gibi alt kelime tabanlı bir tokenizasyondan geçirip bunları token olarak kullanmak. Saf kelime tabanlı yaklaşım, özellikle Türkçede küçük veriyle çok seyrek kalır; ek ve köklere ayırmak büyük olasılıkla daha iyi sonuç verir.

**Soru 10** (derste sözlü, 02:57:00): İngilizce eğitilmiş T5-small, yaklaşık 14 bin örneklik bir Türkçe veriyle ince ayarlanıyor. Bu mantıklı mıdır?

**Cevap (derste verildi):** Hocaya göre çok anlamlı değildir; büyük olasılıkla sıfırdan eğitmekle bir fark olmaz ve veri kümesi o büyüklükte bir model için çok küçük kalır.

**Soru 11** (ekrandan, 02:40:00): Bir n-gram yöntemiyle üretilen 50 cümlenin 29'u anlamlı, 21'i anlamsız bulunmuştur. Başarı oranı nedir? Aynı 50 üçlü için BERT maskelemeyle 24 anlamlı cümle elde edildiyse BERT'in başarı oranı nedir?

**Cevap (derste verildi):** n-gram: $29/50 = 0{,}58$, yani %58. BERT: $24/50 = 0{,}48$, yani %48 (python ile doğrulandı). Bu ölçüte göre n-gram daha başarılıdır.

## Dersin Toparlaması

Dersin ilk yarısında metin işlemenin resimlerle buluştuğu görsel anlambilim ele alındı. Resim tanımı üretme, bir moddan başka bir moda çeviri olarak kodlayıcı-çözücü yapısıyla çözülür: Show and Tell'de önceden eğitilmiş bir CNN resmi gizli uzaydaki bir vektöre çevirir, LSTM bu vektörden tanımı otoregresif olarak, kelime kelime üretir. Önceden eğitilmiş modellerin alt katmanları probleme bağımsız örüntüler taşır ve az veri olduğunda varyansı düşürür. Video tanımlama aynı mantıkla ama daha büyük bir çıkış uzayı ve zaman bilgisi nedeniyle daha zordur. Görsel soru cevaplamada modellerin resme bakmadan soru kalıplarından cevap verebildiği (tenis %41, "2" %39, "evet" %81) görüldü; bu yanlılıklar ve kişi tanıma gibi kısa yollar, modellerin istediğimizi istediğimiz yoldan yapıp yapmadığını sorgulatır. Son olarak etiketsiz veriden ön eğitimli model üreten karşıtsal öğrenme ve SimCLR anlatıldı.

Dersin ikinci yarısında üçüncü ödevin, üç anahtar kelimeden cümle üretimi, sunumları yapıldı. Markov n-gram, LSTM, kısıtlı ışın araması, BERT maske doldurma, GPT-2, mT5 ve FLAN-T5 ince ayarı gibi yöntemler karşılaştırıldı. Genel olarak büyük önceden eğitilmiş Transformer modelleri daha anlamlı cümleler üretti. Hoca karşılaştırmaların sayısal ve insan yargısına dayalı olmasını, eğitimdeki girdi-çıktının açıkça tanımlanmasını, Türkçede alt kelime tokenizasyonu ve FastText kullanılmasını önerdi. Bu öneriler, dersin ilk yarısındaki önceden eğitilmiş model ve veri miktarı tartışmasıyla doğrudan bağlantılıdır.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.